

GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

Lamachaur, 18, Kaski, Nepal



LAB REPORT OF Agile software development

Lab3:Implementation and Exploration of Agile Project Management Tools (Jira, Asana, Trello, ClickUp, and Zoho Projects).

Submitted By:	Submitted To:
Sandip Aryal Roll no: 35	Er.Rajindra Bdr. Thapa

OBJECTIVE

To study and implement the basic features of Agile Project Management tools for planning, organizing, assigning, tracking, and managing software development projects.

THEORY

Agile Project Management is an iterative and incremental approach to managing software development projects. Unlike waterfall models, Agile models emphasizes flexibility, continuous collaboration, customer feedback, and the delivery of working software in small, frequent increments (sprints). Agile teams are self-organizing and cross-functional, and they rely heavily on transparency and communication to succeed.

To effectively implement Agile methodologies such as Scrum and Kanban, teams use specialized project management tools that help organize work, track progress, and facilitate collaboration. These tools provide a centralized platform where all project-related activities from planning to delivery can be managed efficiently.

The following Agile Project Management tools were explored in this experiment:

- 1. Jira:** The industry standard for dev teams, for better or worse. Built for serious Scrum and Kanban workuser stories, sprint planning, burndown charts, the whole deal. It's incredibly flexible, but that flexibility comes with a learning curve. Best if you have someone dedicated to managing it.
- 2. Asana:** Clean, collaborative, and much less intimidating than Jira. You can break projects into tasks and subtasks, assign ownership, set deadlines, and view everything as a list, board, calendar, or timeline. It keeps everyone on the same page without overcomplicating things.
- 3. Trello:** The simplest of the bunch. It's basically a digital Kanban board with cards that move across lists like "To Do" and "Done." Perfect for small teams or personal use. It's lightweight, visual, and does exactly what you need—nothing more, nothing less.
- 4. ClickUp:** The "everything app" that tries to replace Jira, Asana, and a dozen others. It packs in tasks, sprints, docs, time tracking, and automations all in one place. Super configurable, but you'll need patience to set it up just right for your team.

5. Zoho Projects: A solid, no-nonsense tool that plays especially well if you're already in the Zoho ecosystem. Offers Gantt charts, task boards, timesheets, and discussion forums. It gets the job done without too much flash.

OBJECTIVES

- To understand the core concepts and principles of Agile Project Management.
- To explore and compare different Agile Project Management tools including jira, Asana, Trello, ClickUp, and Zoho Projects.
- To create and manage projects, tasks, and user stories using these tools.
- To assign work to team members and set appropriate priorities and deadlines.
- To track project progress using Kanban boards, Scrum boards, and dashboards.
- To understand sprint planning, backlog management, and progress monitoring.

ALGORITHM

1. Select an Agile Project Management tool from the available options (Jira, Asana, Trello, ClickUp, or Zoho Projects).
2. Create a new account or log in to the chosen platform.
3. Create a new project and provide relevant details such as project name, description and methodology (Scrum or Kanban).
4. Define the project backlog by creating tasks, user stories, or work items.
5. Organize tasks into logical groupings using epics, milestones, or categories.
6. Assign each task to specific team members and set priority levels (High, Medium, Low).
7. Set due dates and estimated effort for each task to facilitate planning.
8. Organize tasks on the board by moving them into appropriate columns (e.g., To Do, In Progress, Review, Done).
9. If using Scrum, create a sprint and add selected backlog items to the sprint.
10. Update task statuses regularly as work progresses and team members complete their assignments.
11. Use dashboards, reports, and burndown charts to monitor overall project health and progress.
12. Perform retrospectives or reviews using the tool's collaboration features.
13. Document observations and compare features across different tools.

IMPLEMENTATION

The following activities were performed using an Agile Project Management tool (Trello and Jira were primarily used for hands-on practice, while others were studied conceptually):

- 1. Project Creation:** A new software development project named "Agile Task Manager" was created on the platform.
- 2. Backlog Creation:** Multiple user stories and tasks were added to the project backlog. Each item included a title, description, and acceptance criteria.
- 3. Task Assignment:** Tasks were assigned to different team members based on their roles and expertise.
- 4. Prioritization:** Each task was assigned a priority level to ensure that critical work was addressed first.
- 5. Sprint Planning:** A two-week sprint was planned, and selected backlog items were moved into the sprint.
- 6. Board Management:** Using the Kanban board, tasks were moved from "To Do" to "In Progress," then to "Review," and finally to "Done" as work progressed.
- 7. Progress Tracking:** The dashboard was used to monitor task completion rates, track team workload, and identify bottlenecks.
- 8. Collaboration:** Team members added comments, attached files, and updated task statuses in real-time to maintain transparency.

OUTPUT

The Agile Project Management tool was successfully used to manage project activities. The following key outcomes were observed:

- Project was created successfully with all required configurations.
- Tasks and user stories were added to the backlog effectively.
- Team members were assigned to tasks with appropriate priority levels.
- Task progress was tracked transparently using the Kanban/Scrum board.
- Project status was monitored through real-time dashboards and progress reports.
- Collaboration and communication among team members improved significantly.
- The tools helped in identifying bottlenecks and reprioritizing work efficiently.

RESULT

The implementation of Agile Project Management tools was completed successfully. The tools effectively helped in organizing project tasks, improving team collaboration, monitoring project progress, and managing software development activities in an efficient and structured manner. The hands-on experience with Trello and Jira, along with the conceptual study of Asana, ClickUp, and Zoho Projects, provided a comprehensive understanding of the available Agile tooling landscape.

CONCLUSION

This experiment provided practical and theoretical knowledge of Agile Project Management tools including Jira, Asana, Trello, ClickUp, and Zoho Projects. It is evident that these tools are indispensable for modern Agile software development teams.

To implement and perform unit testing on a JavaScript calculator application using the Jest framework and to study different software testing tools and techniques such as Test Case Generation, JUnit, Selenium, and Load Testing.

TOOLS AND TECHNOLOGIES

- **Nodejs** : A JavaScript runtime environment needed to execute the calculator application and test scripts outside a web browser.
- **npm** : The package manager used to install Jest and manage project dependencies.
- **Jest Framework**: A JavaScript testing framework used to define, execute, and report on unit tests.
- **VS Code / Text Editor**: For writing the source code for the calculator and creating the Jest test suites.
- **Command Line Interface (CLI)**: To initialize the project, run the test commands, and view test execution reports.

THEORY

Software testing is the process of evaluating and verifying that a software product or application does what it is supposed to do. Different levels of testing exist, ranging from unit testing (isolated components) to system and acceptance testing.

Jest is a modern, zero-configuration testing framework primarily used for JavaScript and Node.js applications. It comes with built-in assertion libraries, mocking capabilities, and test runners, making it ideal for Test-Driven Development (TDD).

Alongside Jest, this experiment explores several other critical tools and concepts in the broader scope of software quality assurance:

Test Case Generation: The process of designing test inputs and expected outputs to ensure thorough code coverage. Techniques include Equivalence Partitioning (testing inputs in groups) and Boundary Value Analysis (testing edge cases, such as minimum/maximum values).

JUnit: A widely used testing framework for Java applications. Unlike Jest (JavaScript), JUnit is used for Java-based back-end or Android development, following a similar unit testing philosophy but with different syntax.

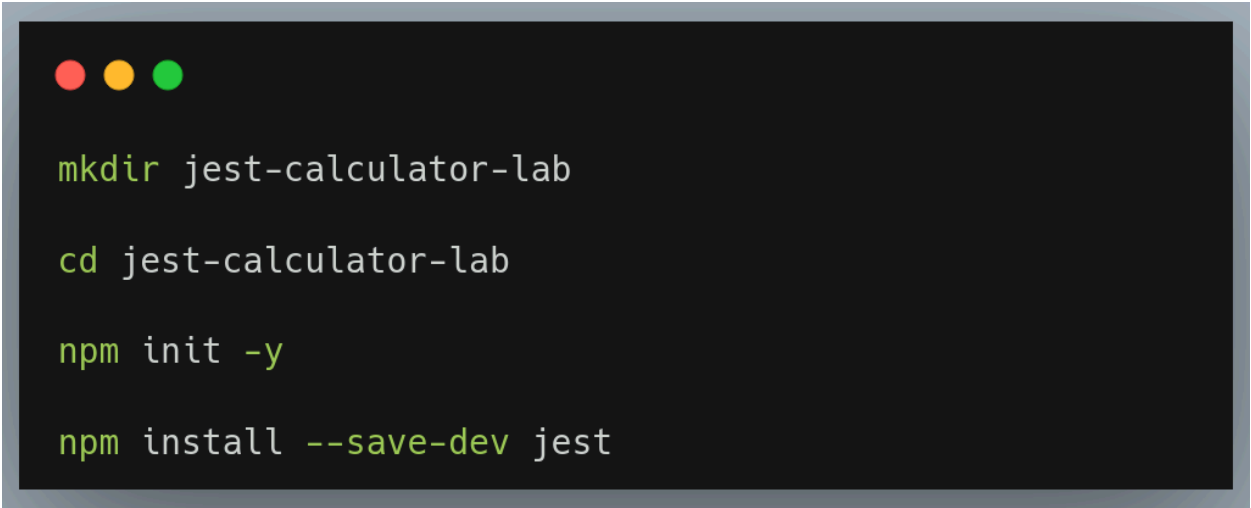
Selenium: A powerful open-source tool specifically designed for automated UI and web application testing. While Jest executes JavaScript logic in the background, Selenium automates real web browsers (like Chrome or Firefox) to simulate user clicks, form inputs, and navigation.

Load Testing: A non-functional testing technique used to simulate real-world usage loads (e.g., thousands of concurrent users) to determine how a system behaves under heavy stress. Tools like JMeter or LoadRunner are commonly used for this.

PROCEDURE

1. Environment Setup:

Create a new project directory and initialize it with npm. Install Jest as a development dependency, and update the `package.json` file to enable Jest testing scripts.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains four lines of text representing terminal commands.

```
mkdir jest-calculator-lab  
cd jest-calculator-lab  
npm init -y  
npm install --save-dev jest
```

Modify the `package.json` scripts block:



```
"scripts": {  
  "test": "jest"  
}
```

2. Writing the Application Logic

Create a source file named `calculator.js` containing the basic arithmetic functions to be tested.



```
// calculator.js

function add(a, b) {
    return a + b;
}

function subtract(a, b) {
    return a - b;
}

function multiply(a, b) {
    return a * b;
}

function divide(a, b) {
    if (b === 0) {
        throw new Error("Division by zero is not allowed");
    }
    return a / b;
}

module.exports = { add, subtract, multiply, divide };
```

3. Generating Test Cases and Writing Test Suites

Create the test file `calculator.test.js`. Here, test cases are generated to cover standard functionality, boundary conditions, and expected error handling.



```
const { add, subtract, multiply, divide } = require('./calculator');

// Standard Unit Tests
test('adds 5 + 10 to equal 15', () => {
  expect(add(5, 10)).toBe(15);
});

test('subtracts 10 - 4 to equal 6', () => {
  expect(subtract(10, 4)).toBe(6);
});

test('multiplies 7 * 8 to equal 56', () => {
  expect(multiply(7, 8)).toBe(56);
});

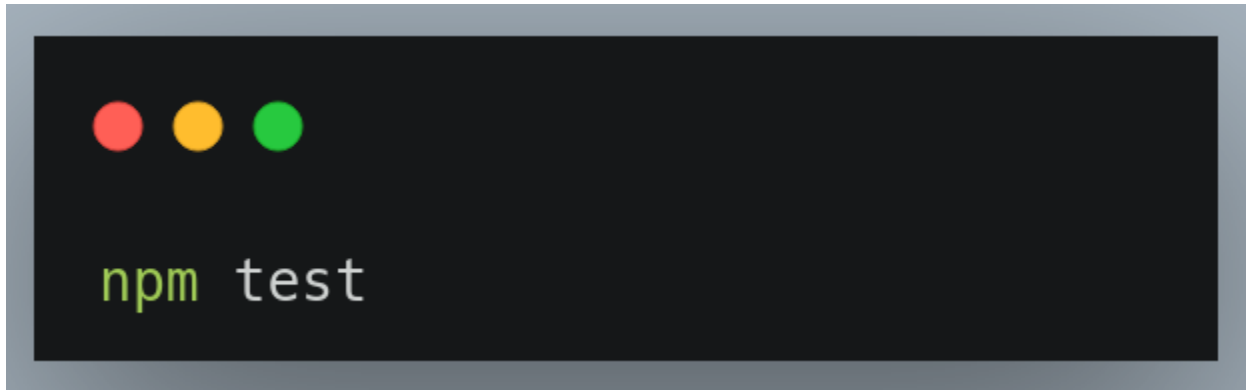
// Boundary and Exception Tests
test('handles division by zero gracefully', () => {
  expect(() => divide(10, 0)).toThrow("Division by zero is not allowed");
});

test('divides 12 by 4 to equal 3', () => {
  expect(divide(12, 4)).toBe(3);
});

// Testing Edge Cases (Boundary Value Analysis)
test('adds negative numbers correctly: -5 + (-3) equals -8', () => {
  expect(add(-5, -3)).toBe(-8);
});

test('multiplies by zero correctly: 10 * 0 equals 0', () => {
  expect(multiply(10, 0)).toBe(0);
});
```

4. **Executing the Test Suite** Run the defined test suite using the Node.js command line. Jest automatically searches for any files ending in `.test.js` or `.spec.js` and runs them sequentially.



RESULT

Upon executing the `npm test` command, Jest scans the project, runs the defined test cases, and provides a comprehensive console report. The output displays a green checkmark for passing tests, alongside a summary of the total tests run, the number passed, and the number failed. If any test fails, Jest provides a detailed diff of the expected versus actual output, allowing for immediate debugging. For this specific calculator implementation, all generated test cases—covering normal operations, negative numbers, zero multiplication, and division-by-zero exceptions—should pass successfully, confirming that the calculator module is mathematically and logically robust.

CONCLUSION

This experiment successfully demonstrated the implementation of unit testing on a JavaScript calculator application using the Jest framework. By generating meaningful test cases based on boundary value analysis and exception handling, the reliability of the basic arithmetic functions was validated.